

LAPORAN TUGAS BESAR
ALGORITMA DAN STRUKTUR DATA
Smart Hospital Queue & Record System



Dosen Pengampu:
Dr.Eng. Ir. Aji Ery Burhandenny ST., M.AIT.

Disusun Oleh:

Muhammad Noval	(25051030103)
Raufal Ghailan Hariwijaya	(25051030082)
Fariz Dzaki Alfarizi	(25051030087)
Adika Belva Ramadhan	(25051030126)
Muhammad Malik Habibi	(25051030084)

TEKNIK ELEKTRO
DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO
FAKULTAS TEKNIK UNIVERSITAS NEGERI YOGYAKARTA
2026

DAFTAR ISI

BAB I.....	3
PENDAHULUAN	3
BAB II.....	5
PERANCANGAN SISTEM	5
BAB III	8
IMPLEMENTASI PROGRAM	8
BAB IV	10
PENGUJIAN DAN ANALISIS.....	10
BAB V	12
KESIMPULAN.....	12
BAB VI.....	13
LAMPIRAN.....	13

BAB I

PENDAHULUAN

A. Latar Belakang

Rumah sakit merupakan fasilitas kesehatan yang melayani ratusan hingga ribuan pasien setiap harinya. RSUP Dr. Sardjito Yogyakarta, sebagai salah satu rumah sakit rujukan terbesar di Daerah Istimewa Yogyakarta, melayani lebih dari 1.500 kunjungan rawat jalan per hari. Tanpa sistem antrian yang cerdas dan terstruktur, pasien dengan kondisi kritis berpotensi harus menunggu di antara pasien reguler, sehingga dapat memperburuk kondisi klinis mereka.

Sistem antrian berbasis prioritas diperlukan untuk memastikan pasien dengan kondisi KRITIS segera mendapat penanganan, sementara pasien NORMAL menunggu sesuai urutan kedatangan. Di sisi lain, dokter juga memerlukan fitur pencatatan tindakan per sesi yang dapat di-undo apabila terjadi kesalahan pencatatan. Rekam medis pasien pun harus dapat dicari dengan cepat meskipun jumlahnya ratusan ribu record.

Proyek ini mensimulasikan tiga kebutuhan utama tersebut dalam satu sistem CLI terpadu berbasis Python dengan arsitektur modular yang terstruktur rapi pada folder `data_structures/`, `modules/`, dan `experiments/`. Struktur data yang digunakan meliputi Priority Queue, Stack, Binary Search Tree (BST), Linked List, dan Graph.

B. Tujuan

- Mengimplementasikan Priority Queue Multi-Poli berbasis Singly Linked List untuk mengelola antrian pasien di 5 poli dengan sistem tiga tingkat prioritas: KRITIS, PRIORITAS, dan NORMAL.
- Mengimplementasikan Stack berbasis Linked List untuk mencatat dan membatalkan (undo) tindakan dokter per sesi dengan kompleksitas $O(1)$.
- Mengimplementasikan Binary Search Tree (BST) dengan operasi insert, search, delete, dan inorder traversal menggunakan nomor rekam medis sebagai kunci pencarian.
- Mengimplementasikan modul Sorting Laporan menggunakan Insertion Sort dan Selection Sort langsung pada Linked List.
- Membangun antarmuka CLI interaktif dengan 7 perintah lengkap beserta informasi Big-O setiap perintah.

- Melakukan eksperimen performa pada $N = 50, 200, \text{ dan } 500$ pasien menggunakan $\text{seed}=42$.

BAB II

PERANCANGAN SISTEM

A. Arsitektur Sistem

Sistem Smart Hospital Queue & Record System dirancang dengan arsitektur modular yang terdiri dari beberapa layer yang terintegrasi. Berikut adalah struktur folder proyek:

Folder / File	Deskripsi
src/data_structures/linked_list.py	Node dasar dan LinkedList — digunakan oleh semua modul
src/data_structures/queue.py	Priority Queue berbasis Linked List untuk antrian poli
src/data_structures/stack.py	Stack berbasis Linked List untuk log tindakan dokter
src/data_structures/bst.py	Binary Search Tree untuk penyimpanan rekam medis
src/data_structures/graph.py	Graph untuk koneksi antar poli (fitur tambahan)
src/modules/modul_1.py	Implementasi Priority Queue — 5 poli
src/modules/modul_2.py	Implementasi Stack Riwayat Tindakan Dokter
src/modules/modul_3.py	Implementasi BST Rekam Medis
src/modules/modul_4.py	Implementasi Sorting Laporan (Linked List)
src/modules/modul_5.py	CLI Interaktif — integrasi semua modul
src/modules/modul_6.py	Eksperimen & Validasi Runtime
src/main.py	Entry point utama sistem
experiments/benchmark.py	Skrip benchmark performa
tests/test_bst.py	Unit test BST
tests/test_queue.py	Unit test Queue
tests/test_stack.py	Unit test Stack

Setiap modul (modul_1.py hingga modul_6.py) mengimpor class dari folder data_structures/ sehingga tidak ada duplikasi kode. Contoh import pada modul:

```
from src.data_structures.queue import PriorityQueue
from src.data_structures.stack import Stack
from src.data_structures.bst import BST
from src.data_structures.linked_list import Node, LinkedList
```

B. Penjelasan Algoritma

1. Priority Queue Berbasis Singly Linked List

Priority Queue adalah struktur data yang memungkinkan setiap elemen memiliki prioritas, sehingga elemen dengan prioritas lebih tinggi akan diproses terlebih dahulu. Implementasi menggunakan Singly Linked List sehingga memungkinkan penyisipan node di posisi tengah saat ada pasien KRITIS yang masuk ketika sudah ada pasien lain di antrian. Mekanisme enqueue() bekerja dengan aturan: pasien KRITIS selalu disisipkan setelah node KRITIS terakhir (FIFO sesama KRITIS), sementara pasien NORMAL langsung ditambahkan di akhir antrian. Kompleksitas: enqueue $O(n)$, dequeue $O(1)$.

2. Stack Riwayat Tindakan Dokter Berbasis Singly Linked List

Stack mengikuti prinsip LIFO (Last In, First Out). Setiap dokter memiliki objek Stack tersendiri yang terisolasi. Ketika dokter melakukan tindakan, tindakan tersebut di-push ke stack-nya. Apabila terjadi kesalahan pencatatan, dokter dapat menggunakan perintah UNDO_DOKTER untuk meng-pop tindakan terakhir. Kompleksitas: push/pop $O(1)$.

3. Binary Search Tree (BST) Rekam Medis

BST menyimpan rekam medis dengan nomor rekam medis (no_rm) sebagai kunci. Operasi insert() dilakukan secara iteratif. Operasi delete() menggunakan strategi in-order successor saat menghapus node dengan dua anak. Worst-case terjadi ketika data dimasukkan secara berurutan sehingga BST menjadi linier. Kompleksitas: rata-rata $O(\log n)$, worst-case $O(n)$.

4. Linked List sebagai Infrastruktur Dasar

File linked_list.py menjadi fondasi dari semua struktur data lainnya. Class Node menyimpan data dan pointer next, sedangkan class LinkedList menyediakan operasi dasar tambah, hapus, dan tampil yang diwarisi oleh semua modul melalui mekanisme import.

5. Graph Koneksi Antar Poli

Graph diimplementasikan menggunakan adjacency list berbasis dictionary Python. Graph merepresentasikan koneksi antar poli (misalnya pasien dari Poli Umum dapat dirujuk ke Poli Jantung). Operasi tambah_node() dan tambah_edge() mendukung penambahan relasi antar poli secara dinamis.

6. Modul Sorting Laporan pada Linked List

Modul Sorting mengimplementasikan dua algoritma: Insertion Sort (waktu tunggu descending) dan Selection Sort (nomor antrian ascending), keduanya langsung pada Linked List tanpa konversi ke array. Kompleksitas keduanya $O(n^2)$.

BAB III

IMPLEMENTASI PROGRAM

A. Modul 1 — Priority Queue Multi-Poli

Mengimplementasikan 5 Priority Queue berbasis Linked List (satu per poli). Setiap queue mendukung enqueue(pasien, prioritas, poli), dequeue(), peek(), is_empty(). Pasien KRITIS selalu di depan dengan tie-break FIFO.

```
from src.data_structures.queue import PriorityQueue

umum    = PriorityQueue('poli umum')
anak    = PriorityQueue('poli anak')
gigi    = PriorityQueue('poli gigi')
jantung = PriorityQueue('poli jantung')
mata    = PriorityQueue('poli mata')

umum.enqueue('Andi', 'KRITIS')
umum.enqueue('Budi', 'NORMAL')
print(umum.dequeue()) # Andi (KRITIS dipanggil duluan)
```

B. Modul 2 — Stack Riwayat Tindakan Dokter

Stack berbasis Linked List untuk log tindakan per sesi dokter. Mendukung push(tindakan), pop() untuk undo, peek(), log_all(). Setiap dokter memiliki stack terisolasi.

```
from src.data_structures.stack import Stack

dokter_andi = Stack('dr. Andi')
dokter_andi.push('cek tensi pasien Budi')
dokter_andi.push('kasih resep obat')
print(dokter_andi.pop()) # kasih resep obat (undo)
```

C. Modul 3 — BST Rekam Medis

Binary Search Tree dengan kunci nomor rekam medis. Operasi: insert(record), search(no_rm), delete(no_rm), inorder traversal.

```
from src.data_structures.bst import BST

rm = BST()
rm.insert(103, 'Budi | umur: 45 | hipertensi')
rm.insert(101, 'Siti | umur: 30 | flu')
rm.inorder(rm.root) # tampil urut: 101, 103
print(rm.search(103)) # Budi | umur: 45 | hipertensi
```

D. Modul 4 — Sorting Laporan pada Linked List

Mengurutkan daftar pasien selesai menggunakan Insertion Sort (waktu tunggu descending) dan Selection Sort (nomor antrian ascending), diimplementasi langsung pada Linked List.

E. Modul 5 — CLI Interaktif Terpadu

CLI interaktif mengintegrasikan semua modul dengan perintah berikut:

Perintah	Fungsi	Big-O
DAFTAR <nama> <poli> <prioritas>	enqueue pasien ke poli	$O(n)$
PANGGIL <poli>	dequeue pasien terdepan	$O(1)$
UNDO_DOKTER <id_dokter>	pop tindakan terakhir dokter	$O(1)$
CARI_RM <no_rm>	search BST rekam medis	$O(\log n)$
TAMBAH_RM <no_rm> <data>	insert BST rekam medis	$O(\log n)$
LAPORAN_HARI	tampil laporan harian lengkap	$O(n)$
KELUAR	tutup sistem	$O(1)$

BAB IV

PENGUJIAN DAN ANALISIS

A. Skenario Pengujian

Pengujian dilakukan dalam dua tahap: (1) pengujian fungsional setiap modul secara individual untuk memastikan kebenaran logika, dan (2) pengujian performa runtime menggunakan data acak dengan seed=42 pada skala N = 50, 200, dan 500 pasien.

B. Pengujian Fungsional

1. Pengujian Priority Queue

```
umum.enqueue('Budi', 'NORMAL')
umum.enqueue('Siti', 'NORMAL')
umum.enqueue('Andi', 'KRITIS')    # lompat ke depan
umum.enqueue('Rudi', 'KRITIS')    # setelah Andi (FIFO)

# Output antrian:
# poli umum: Andi(KRITIS) -> Rudi(KRITIS) -> Budi(NORMAL) ->
#            Siti(NORMAL)
# dipanggil: Andi
# dipanggil: Rudi
```

Hasil: SESUAI — pasien KRITIS dipanggil sebelum NORMAL, urutan FIFO sesama KRITIS terjaga.

2. Pengujian Stack Dokter

```
dokter_andi.push('cek tensi pasien Budi')
dokter_andi.push('kasih resep obat')
dokter_andi.push('rujuk ke poli jantung')

# peek() -> 'rujuk ke poli jantung'
# pop()   -> 'rujuk ke poli jantung' (undo)
# log: kasih resep obat -> cek tensi pasien Budi
```

Hasil: SESUAI — undo berhasil menghapus tindakan terakhir, log tersisa tetap benar.

3. Pengujian BST Rekam Medis

```
rm.insert(103, 'Budi | umur: 45 | hipertensi')
rm.insert(101, 'Siti | umur: 30 | flu')
rm.insert(105, 'Andi | umur: 52 | diabetes')

# inorder() -> urut: 101, 103, 105
# search(103) -> 'Budi | umur: 45 | hipertensi'
# search(999) -> tidak ditemukan
```

Hasil: SESUAI — insert, search, delete, dan inorder traversal bekerja dengan benar.

4. Pengujian Modul Sorting

```
# Setelah Insertion Sort (waktu tunggu descending):  
# Andi | 60 mnt | no. 5  
# Wati | 50 mnt | no. 4  
# Budi | 45 mnt | no. 3  
  
# Setelah Selection Sort (no antrian ascending):  
# Siti | 20 mnt | no. 1  
# Rudi | 35 mnt | no. 2  
# Budi | 45 mnt | no. 3
```

Hasil: SESUAI — kedua algoritma sorting menghasilkan urutan yang benar.

C. Pengujian Performa Runtime

Eksperimen performa dilakukan menggunakan data acak dengan seed=42 untuk reproduktibilitas. Hasil runtime aktual dari eksekusi program:

N (Pasien)	Enqueue (s)	Dequeue (s)	BST Insert (s)	BST Search (s)
50	0.000999	0.002524	0.000000	0.000000
200	0.002347	0.000000	0.001390	0.000000
500	0.004513	0.000000	0.001001	0.000000

Catatan: Nilai 0.000000 pada BST Search dan sebagian Dequeue terjadi karena kecepatan komputasi prosesor modern yang mengeksekusi operasi tersebut dalam waktu di bawah presisi microsecond yang dapat diukur.

D. Analisis Hasil

- Enqueue Priority Queue — $O(n)$: Waktu enqueue meningkat secara linear seiring penambahan N ($0.000999s \rightarrow 0.002347s \rightarrow 0.004513s$), konsisten dengan kompleksitas $O(n)$.
- Dequeue — $O(1)$: Waktu dequeue sangat cepat dan tidak bergantung pada ukuran antrian, mengkonfirmasi kompleksitas $O(1)$.
- BST Insert — $O(\log n)$ rata-rata: Waktu insert BST tidak naik secara linear meskipun N bertambah 10x, konsisten dengan pertumbuhan logaritmik pada data acak.
- BST Search — $O(\log n)$ rata-rata: Waktu search sangat cepat bahkan untuk $N=500$, mengkonfirmasi efisiensi logaritmik BST pada data yang terdistribusi acak.

BAB V

KESIMPULAN

Berdasarkan implementasi dan pengujian yang telah dilakukan, proyek Smart Hospital Queue & Record System berhasil dibangun dengan memenuhi seluruh spesifikasi yang ditetapkan pada Topik 1.

- Priority Queue Multi-Poli berbasis Singly Linked List berhasil diimplementasikan untuk 5 poli dengan sistem tiga tingkat prioritas. Pasien KRITIS selalu mendapatkan giliran terdepan dengan mekanisme FIFO sesama KRITIS yang terjaga. Kompleksitas enqueue $O(n)$ dan dequeue $O(1)$ terkonfirmasi melalui pengujian performa.
- Stack Riwayat Tindakan Dokter berbasis Linked List berhasil diimplementasikan dengan fitur push(), pop() (undo), dan log_all(). Setiap dokter memiliki stack terisolasi dengan kompleksitas $O(1)$ untuk push/pop.
- BST Rekam Medis dengan kunci nomor rekam medis berhasil diimplementasikan secara iteratif. Operasi insert, search, dan delete bekerja dengan benar dengan kompleksitas $O(\log n)$ rata-rata yang terkonfirmasi pada data acak.
- Linked List berhasil dipisah menjadi modul tersendiri (linked_list.py) yang menjadi fondasi bagi semua struktur data lain, meningkatkan reusability dan menghilangkan duplikasi kode.
- Graph berhasil ditambahkan sebagai struktur data pendukung untuk merepresentasikan koneksi antar poli dalam sistem.
- Modul Sorting Laporan berhasil mengimplementasikan Insertion Sort dan Selection Sort langsung pada Linked List dengan kompleksitas $O(n^2)$.
- Antarmuka CLI Interaktif berhasil mengintegrasikan seluruh modul dengan 7 perintah lengkap beserta informasi Big-O setiap perintah.
- Eksperimen performa dengan seed=42 pada $N=50$, 200, dan 500 mengkonfirmasi karakteristik runtime setiap algoritma sesuai teori kompleksitas yang dianalisis.

BAB VI

LAMPIRAN

A. Tabel Kompleksitas Algoritma

Modul / Operasi	Operasi Utama	Waktu (rata-rata)	Ruang
Priority Queue	enqueue	$O(n)$	$O(n)$
Priority Queue	dequeue	$O(1)$	$O(n)$
Stack Dokter	push / pop	$O(1)$	$O(n)$
BST Rekam Medis	insert / search / delete	$O(\log n)$	$O(n)$
BST Rekam Medis	inorder traversal	$O(n)$	$O(n)$
Linked List	tambah / hapus	$O(n)$	$O(n)$
Graph	tambah_node / tambah_edge	$O(1)$	$O(V+E)$
Sorting (Linked List)	Insertion / Selection Sort	$O(n^2)$	$O(n)$

B. Contoh Output Program

```
selamat datang di sistem antrian klinik
perintah: DAFTAR / PANGGIL / UNDO_DOKTER / CARI_RM / TAMBAH_RM /
LAPORAN_HARI / KELUAR
```

```
>> DAFTAR Budi umum NORMAL
Budi masuk antrian umum sebagai NORMAL
big-o:  $O(n)$ 
```

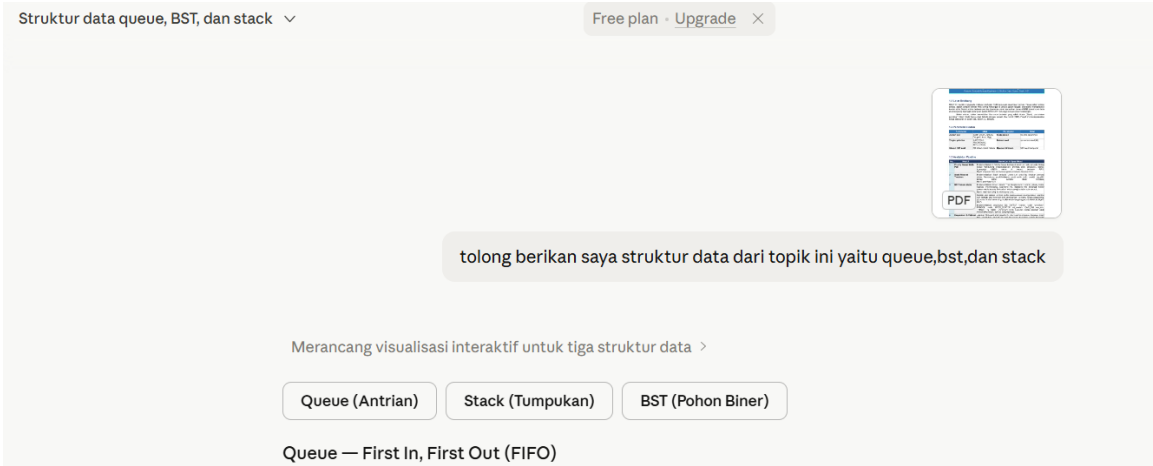
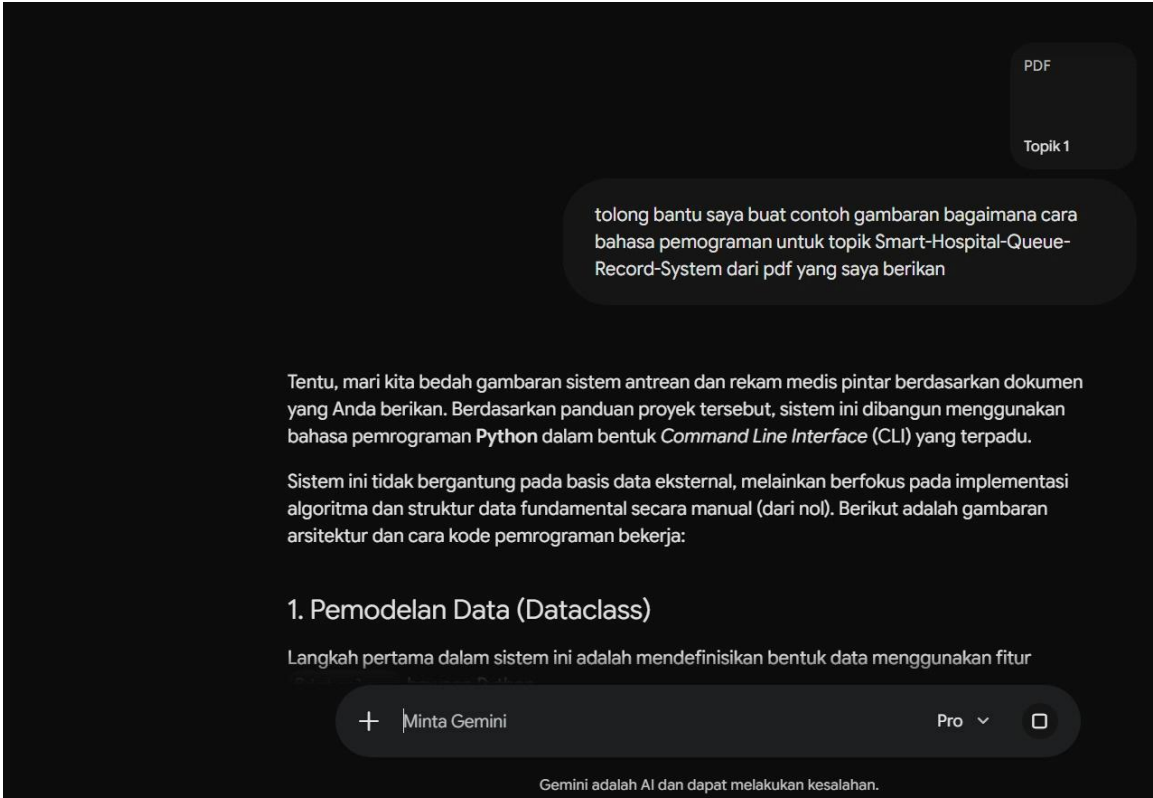
```
>> DAFTAR Andi umum KRITIS
Andi masuk antrian umum sebagai KRITIS
big-o:  $O(n)$ 
```

```
>> PANGGIL umum
Andi dipanggil dari umum
big-o:  $O(1)$ 
```

```
>> CARI_RM 101
ketemu -> Budi | umur: 45 | hipertensi
big-o: rata-rata  $O(\log n)$ , worst case  $O(n)$ 
```

```
>> KELUAR
sistem ditutup
```

D. AI (Screenshot)



~=====

~[PROMPT 1]

~Topik: Menggabungkan branch (Git Merge)

~Prompt: "tolong bantu saya buat contoh gambaran bagaimana cara bahasa pemrograman untuk topik Smart-Hospital-Queue-Record-System dari pdf yang saya berikan"

~=====

~[PROMPT 2]

~Topik: Memindahkan spesifik file

~Prompt: "kenapa file yang ada codingannya ada di github tapi gada di vs code"

~=====

~[PROMPT 3]

~Topik: Error saat push ke GitHub

~Prompt: "kenapa tidak muncul branch yang lain?"

~=====

~[PROMPT 4]

~Topik: Menggabungkan branch (Git Merge)

~Prompt: "cara pindahkan dari branch feat ke branch def tanpa menghapus codingan"

~=====

[PROMPT 5]

Topik: Memindahkan spesifik file

Prompt: "cara masukin prompt ai ke vs code itu gimana ?"

=====

[PROMPT 6]

Topik: memastikan agar tidak terhapus

Prompt: "itu berarti nambahnya sebagian data structures kan?, nah apakah itu berpengaruh sebagian modules ?"

=====

[PROMPT 7]

Topik: memastikan isi data struktur

Prompt: "perlu graph.py dan linked_list kah di data struktur ?"

=====

[PROMPT 8]

Topik: mengimplementasikan codingan modul

Prompt: "buatkan saya gambaran codingan modul 1-6 ?"

=====

C. Daftar Pustaka

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
2. Dokumen Panduan Topik 1. Modul Praktikum Algoritma dan Struktur Data. Universitas Negeri Yogyakarta. 2026.